

Đại Học Công Nghiệp Thành Phố Hồ Chí Minh

Môn học : Thực Hành TK Vi Mạch Số với HDL

BÀI 5: CLOCK VÀ ĐỊNH THỜI

BÀI 4: MẠCH ĐẾM

1

Giới thiệu xung Clock trên KIT DE2

2

Cách chia tần số

3

Mô tả mạch số dùng xung **CLOCK_50**

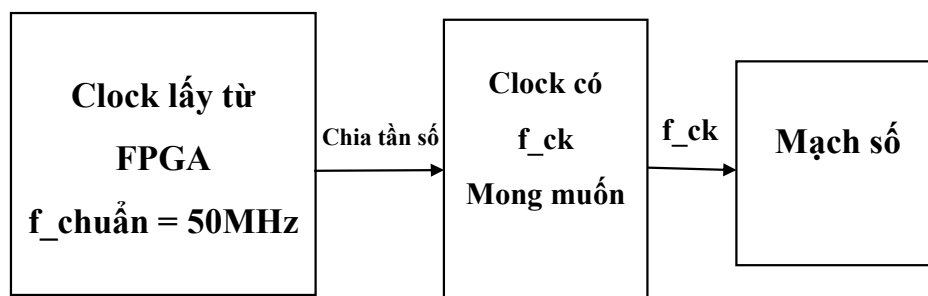
1. Giới thiệu xung CLOCK trên KIT DE2

Kit DE2 cung cấp hai nguồn tín hiệu xung Clock 27 MHz và 50 MHz. Ngoài ra ta cũng có thể cung cấp nguồn xung Clock từ bên ngoài thông qua cổng ngõ vào SMA. Sơ đồ mạch kết nối nguồn xung Clock và danh sách liệt kê những tín hiệu nguồn xung Clock kết nối đến những Pin tương ứng của Cyclone II FPGA.

Signal Name	FPGA Pin No.	Description
CLOCK_27	PIN_D13	27 MHz clock input
CLOCK_50	PIN_N2	50 MHz clock input
EXT_CLOCK	PIN_P26	External (SMA) clock input

2. Chia tần số

- Chia tần số là một công đoạn rất quan trọng trong bất cứ mạch số nào liên quan đến sự đồng bộ và thời gian thực. Trong một mạch số chúng ta cần dùng nhiều tần số khác nhau thì việc chia tần số chính của board FPGA (thường là 50MHz) ra nhiều tần số khác nhau là hết sức cần thiết.
- Cách chia tần số:



Ví dụ 1: tạo xung clock có tần số $f = 1\text{Hz}$

Tính toán giá trị đếm:

Ta có : $f_{\text{chuan}} = 50\text{ MHz}$

$$f_{\text{ck}} = 1\text{ Hz} \rightarrow f_{\text{chuan}} = 50.10^6 f_{\text{ck}} \rightarrow T_{\text{ck}} = 50.10^6 T_{\text{chuan}}$$

→ Vậy cứ 50.10^6 chu kỳ của T_{chuan} thì sẽ tạo thành 1 chu kỳ của T_{ck}

Chọn $T_{\text{on}} = T_{\text{off}} = 25.10^6$

→ Số đếm là 25.10^6 (đếm từ 0 đến 25.10^6 thì xung đảo trạng thái 1 lần)

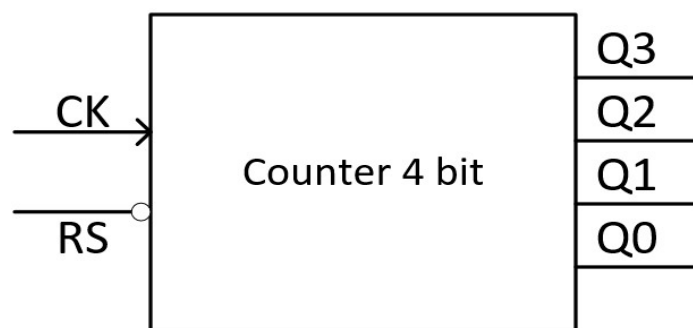
Viết code verilog tạo xung clock có tần số $f = 1\text{Hz}$

```
module tanso_1Hz (CLOCK_50, clock_1s);  
    input CLOCK_50;  
    output reg clock_1s = 1'b0;  
    integer q;  
  
    always @ (posedge CLOCK_50)  
    begin  
        q=q+1;  
        if (q == 25000000)  
        begin  
            clock_1s = ~clock_1s;  
            q = 0;  
        end  
    end  
endmodule
```

3. Mô tả mạch số dùng xung CLOCK_50

Ví dụ 2: viết code mô tả mạch đếm nhị phân 4 bit, mỗi trạng thái đếm tồn tại trong 1s ($T = 1s$), chân RS đồng bộ, tích cực mức thấp

Giải:



Gán chân:

CK : CLOCK_50

RS : SW[0]

Q3 : LEDR[3]

Q2 : LEDR[2]

Q1 : LEDR[1]

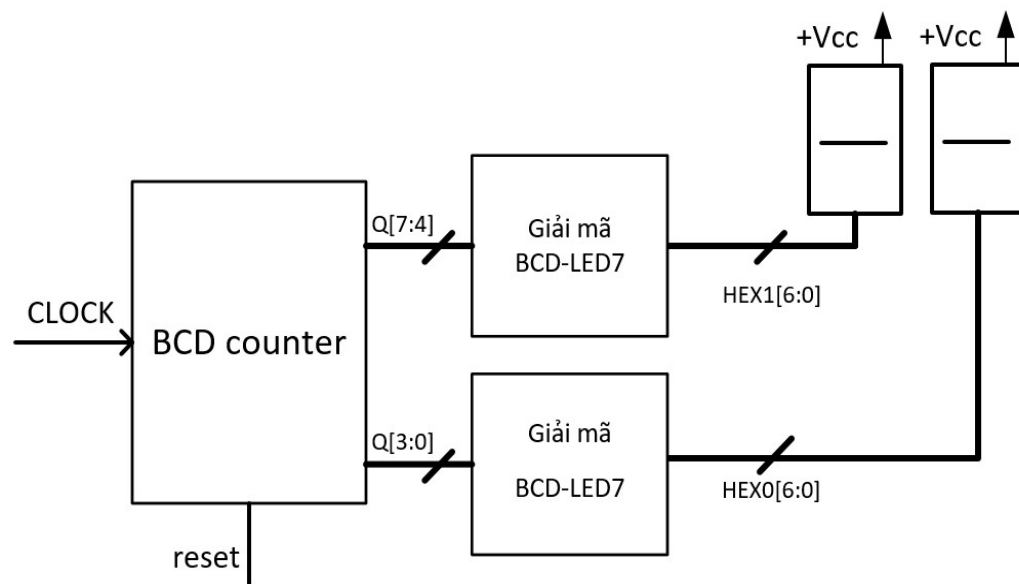
Q0 : LEDR[0]

```
module delay_1s (CLOCK_50,LEDR,SW);
    input [0:0]SW;
    input CLOCK_50;
    output [3:0]LEDR;
    integer q;
    reg [3:0]LEDR;
    reg clock_1s = 1'b0;
    always @ (posedge CLOCK_50)
        begin
            q=q+1;
            if (q == 25000000)
                begin
                    clock_1s = ~clock_1s;
                    q = 0;
                end
        end
end
```

```
        always @ (posedge clock_1s)
        begin
            if (SW[0] == 0)
                LEDR = 4'b0000;
            else
                LEDR = LEDR + 1;
        end
    endmodule
```


Ví dụ 3: Thiết kế mạch đếm BCD đếm từ 00 đến 99, mỗi trạng thái đếm tồn tại trong 1s ($T = 1s$), chân RS đồng bộ, tích cực mức cao. Kết quả hiển thị trên 2 LED 7 đoạn HEX0 và HEX1.

Giải:



Gán chân:

CLOCK : CLOCK_50

reset : SW[0]

```
module aa (CLOCK_50,SW,HEX1,HEX0);  
input CLOCK_50;  
input [0:0]SW;  
output [6:0] HEX1,HEX0;  
wire [3:0] ch, dv;  
reg [3:0] bcd1, bcd2;  
integer q;  
reg clock_1s = 1'b0;  
module aa (CLOCK_50,SW,HEX1,HEX0);  
input CLOCK_50;  
input [0:0]SW;  
output [6:0] HEX1,HEX0;  
wire [3:0] ch, dv;  
reg [3:0] bcd1, bcd2;  
integer q;  
reg clock_1s = 1'b0;
```

```
always @ (posedge clock_1s)  
begin  
    if (SW == 1)  
    begin  
        bcd1 <= 4'b0000;  
        bcd2 <= 4'b0000;  
    end  
    else  
    begin  
        bcd1 <= bcd1 + 1;  
        if (bcd1 == 4'b1001)  
        begin  
            bcd1 <= 4'b0000;  
            bcd2 <= bcd2 + 1;  
            if (bcd2 == 4'b1001) bcd2 <= 4'b0000;  
        end  
    end  
end
```

```
assign ch = bcd2; //chuyen kieu reg sang kieu wire
assign dv = bcd1;
bcd7seg(dv,HEX0);
bcd7seg(ch,HEX1);
endmodule
// mach giai ma led bay doan
module bcd7seg(bcd,display);
input [3:0]bcd;
output [6:0]display;
reg [6:0]display;
always@(bcd) begin
case(bcd)
4'b0000: display=7'b1000000;
4'b0001: display=7'b1111001;
4'b0010: display=7'b0100100;
4'b0011: display=7'b0110000;
```

```
4'b0100: display=7'b0011001;
4'b0101: display=7'b0010010;
4'b0110: display=7'b0000010;
4'b0111: display=7'b1111000;
4'b1000: display=7'b0000000;
4'b1001: display=7'b0010000;
default display = 7'b1111111;
endcase
end
Endmodule
```

Cảm ơn các bạn đã lắng nghe!

